

Connecting Claude Code to Local n8n with n8n-MCP

A complete, step-by-step tutorial — from an empty terminal to a working workflow that Claude Code builds for you.

Everything in this guide uses free software and free service tiers. No paid plan, no credit card, no cloud hosting.

Part 0 — What you are building

By the end of this guide you will have:

1. **n8n running locally** at `http://localhost:5678`.
2. **The n8n-MCP server wired into Claude Code**, so Claude can read n8n's node documentation *and* create, validate, and run workflows on your instance.
3. **Google Sheets and Gmail connected to n8n** with real OAuth credentials — set up properly, with every failure mode explained.
4. **A working 5-node workflow**, built by asking Claude Code for it in plain English.

The finished workflow reads a list of contacts from a Google Sheet, sends each one a personalised email through Gmail, and writes the result back to the sheet so nobody gets emailed twice:

```
Manual Trigger
-> Google Sheets (read the Contacts tab)
-> If            (only rows not yet sent)
-> Gmail         (send a personalised message)
-> Google Sheets (mark the row as sent)
```

That is a genuinely useful automation, and it is small enough to understand completely.

Why bother connecting Claude Code to n8n?

n8n has around 2,700 nodes. Each node has dozens of parameters, and the parameter names change between versions. Building a workflow by hand means a lot of clicking, and building one by asking a chatbot from memory means a lot of quietly wrong parameter names that validate fine and then do nothing at runtime.

The n8n-MCP server closes that gap. It gives Claude Code the **live schema** of every node on your instance, plus tools to create and validate workflows against your actual n8n. Claude stops guessing and starts reading.

Prerequisites

You need	Why	Check with
Node.js 20 or newer	Runs both n8n and the MCP server	<code>node -v</code>
Claude Code	The client we are connecting	<code>claude --version</code>
A Google account	For the Sheets and Gmail example	—
A terminal you are comfortable in	Everything here is CLI plus browser	—

You do **not** need Docker, though this guide shows a Docker option too.

Versions this guide was written against

Component	Version
n8n	2.38.1
n8n-mcp	2.82.1
n8n-mcp-skills plugin	1.34.0

n8n moves fast. If a menu name here does not match what you see, trust your screen — the concepts below are stable even when the buttons move.

Part 1 — Run n8n locally

There are two sensible ways to run n8n on your own machine. Pick one.

Option A — npx (fastest, good for learning)

```
npx n8n
```

That is the whole thing. The first run downloads n8n (a few minutes, it is a big package), then prints:

```
Editor is now accessible via:
http://localhost:5678
```

Open that URL. n8n asks you to create an **owner account** — an email and password. This is a local account stored on your own machine; it is not sent anywhere. Use a real email address for it, because password recovery is not configured on a local instance.

Leave that terminal running. n8n stops when you close it.

Where your data lives: `~/.n8n` — workflows, credentials, and an SQLite database. Deleting that folder resets everything, so it is also your backup target.

Option B — Docker Compose (survives reboots, better for real use)

Create `docker-compose.yml`:

```
services:
  n8n:
    image: docker.n8n.io/n8nio/n8n:latest
    container_name: n8n
    restart: unless-stopped
    ports:
      - "5678:5678"
    environment:
      - GENERIC_TIMEZONE=Asia/Karachi
      - TZ=Asia/Karachi
      - N8N_RUNNERS_ENABLED=true
      - N8N_DIAGNOSTICS_ENABLED=false
      - N8N_HOST=localhost
      - N8N_PORT=5678
      - WEBHOOK_URL=http://localhost:5678/
      - N8N_ENCRYPTION_KEY=replace-with-a-long-random-string
    volumes:
      - n8n_data:/home/node/.n8n

volumes:
  n8n_data:
```

Then:

```
docker compose up -d
docker compose logs -f n8n
```

Three of those environment variables matter more than they look:

- **N8N_ENCRYPTION_KEY** encrypts your stored credentials. Set it once, to a long random string, and never change it. If you change it, every credential you have saved becomes permanently unreadable and has to be re-entered by hand. Generate one with `openssl rand -hex 32`.
- **GENERIC_TIMEZONE** is what Schedule Trigger nodes use. Leave it unset and your "9am daily" workflow fires at 9am UTC.
- **WEBHOOK_URL** is the base address n8n advertises for webhooks *and* for OAuth callbacks. Part 7 depends on this being right.

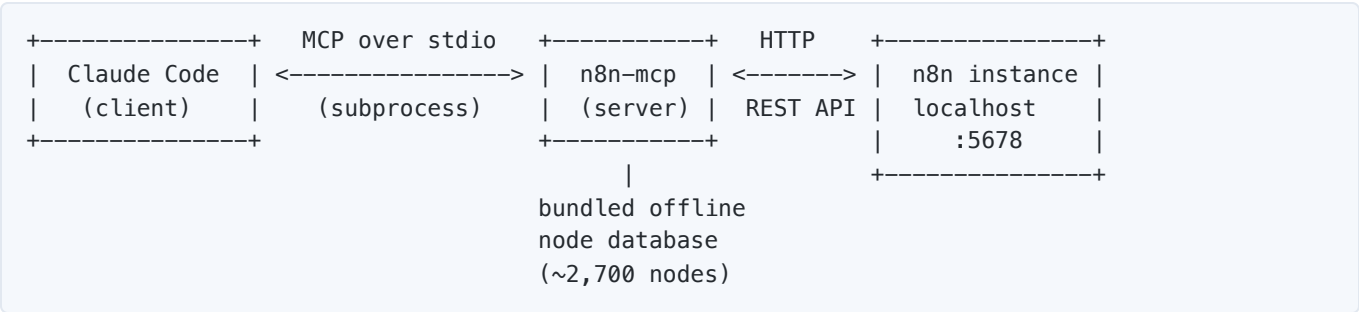
Confirm it is up

```
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:5678/healthz
```

200 means n8n is running and ready.

Part 2 — How the pieces fit together

Before configuring anything, it is worth 60 seconds to understand what talks to what. Almost every setup problem people hit is a misunderstanding of this diagram.



Three things are worth noticing.

n8n-mcp is not a service you install and leave running. Claude Code launches it as a child process when a session starts, talks to it over stdin/stdout, and kills it when the session ends. There is no port, no daemon, nothing to restart.

n8n-mcp carries its own copy of the node documentation. Node schemas, properties, operations, and a library of workflow templates are bundled into the package. That part works with no n8n instance at all.

Only some of the tools need your n8n. This is the single most important idea in this guide, so here it is as a table:

Tool tier	Examples	Needs n8n API?	When available
Documentation & validation	<code>search_nodes</code> , <code>get_node</code> , <code>validate_workflow</code> , <code>search_templates</code>	No	Always
Instance management	<code>n8n_create_workflow</code> , <code>n8n_get_workflow</code> , <code>n8n_test_workflow</code> , <code>n8n_health_check</code>	Yes	Only when <code>N8N_API_URL</code> + <code>N8N_API_KEY</code> are set

So if you connect the server and Claude can look up nodes but says it cannot create a workflow, **nothing is broken**. You have the first tier and not the second, and the fix is an API key. That is Part 3.

Part 3 — Create an n8n API key

The n8n Public API is what n8n-mcp uses to manage workflows. On a self-hosted instance it is enabled by default, but it always requires a key.

Check that the API is enabled

```
curl -s -o /dev/null -w '%{http_code}\n' \
http://localhost:5678/api/v1/workflows
```

Read the result carefully:

Code	Meaning
401	Good. The API is live and asking for a key. This is what you want here.
404	The API is disabled, or your base URL is wrong.
000	n8n is not running or not on port 5678.

A 401 is success at this stage. You have no key yet, so being refused is correct.

Create the key

1. Open `http://localhost:5678` .
2. Click your **user avatar**, bottom-left, then **Settings**.
3. Choose **n8n API** in the settings sidebar.
4. Click **Create an API key**.
5. Give it a label (`claude-code`), pick an expiry, and leave the scopes at their defaults unless you want to restrict them.
6. **Copy the key now.** n8n shows it exactly once. If you lose it, delete the key and make a new one — there is no way to view it again.

Verify the key before you configure anything

Test it in your terminal first. Debugging a bad key through an MCP client is miserable; debugging it with `curl` takes five seconds.

```
export N8N_API_KEY='paste-your-key-here'

curl -s -o /dev/null -w '%{http_code}\n' \
-H "X-N8N-API-KEY: $N8N_API_KEY" \
http://localhost:5678/api/v1/workflows
```

Code	Meaning
200	The key works. Move on to Part 4.
401	Key is wrong, truncated, or expired. Copy it again.
404	Wrong URL — check the port and that there is no <code>/</code> on the end.

Handle this key like a password. It grants full control of your n8n instance: reading credentials metadata, creating workflows, and running them. Do not paste it into a chat, a screenshot, a public repository, or a workflow node field.

Part 4 — Connect n8n-MCP to Claude Code

Now we hand Claude the key. There are two ways; they produce the same result.

Route 1 — the `claude mcp add` command

```
claude mcp add n8n-mcp --scope project \
  -e N8N_API_URL=http://localhost:5678 \
  -e N8N_API_KEY=paste-your-key-here \
  -- npx -y n8n-mcp
```

Reading that command left to right:

Piece	What it does
<code>n8n-mcp</code>	The name this server gets. Tools appear as <code>mcp__n8n-mcp__*</code> .
<code>--scope project</code>	Saves to <code>.mcp.json</code> in this project. Use <code>user</code> for all projects.
<code>-e KEY=value</code>	Environment variables passed to the server process.
<code>--</code>	Ends Claude's own flags. Everything after is the command to run.
<code>npx -y n8n-mcp</code>	Downloads and runs the server. <code>-y</code> skips the install prompt.

The `--` matters. Without it, Claude Code tries to interpret `npx`'s arguments as its own and the command fails in a confusing way.

Scope choice: `--scope project` writes a `.mcp.json` that lives in your repository — convenient for a team, but see the warning below about the key. `--scope user` writes to your personal config instead and is the safer default for a machine-local n8n.

Route 2 — write `.mcp.json` by hand

Same thing, as a file in your project root:

```
{
  "mcpServers": {
    "n8n-mcp": {
      "command": "npx",
      "args": ["-y", "n8n-mcp"],
      "env": {
        "N8N_API_URL": "http://localhost:5678",
        "N8N_API_KEY": "${N8N_API_KEY}",
        "MCP_MODE": "stdio",
        "LOG_LEVEL": "error",
        "DISABLE_CONSOLE_OUTPUT": "true"
      }
    }
  }
}
```

Note `"${N8N_API_KEY}"` . Claude Code expands environment variables in this file, so the key stays in your shell profile and out of version control. Export it before starting Claude:

```
export N8N_API_KEY='paste-your-key-here'
```

The last three variables are not decoration:

- `MCP_MODE=stdio` tells the server which transport to use.
- `LOG_LEVEL=error` and `DISABLE_CONSOLE_OUTPUT=true` stop the server from writing chatty logs to stdout. On a stdio server, stdout is the protocol channel — a stray log line corrupts the JSON-RPC stream and the connection dies with an unhelpful parse error.

If you commit `.mcp.json` , never put the literal key in it. Use the `${VAR}` form above, and add a `.gitignore` entry for any file that does hold secrets. A leaked n8n key is a leaked instance.

Verify the connection

First, from the terminal:

```
claude mcp list
```

You want to see `n8n-mcp` with a connected status. If it says failed, jump to the troubleshooting table below.

Then **restart Claude Code** and open a session in that directory. MCP servers are launched at session start, so a server added mid-session will not appear until you restart. Type `/mcp` to see the server and its tools.

Finally, ask Claude to run its health check:

```
Run n8n_health_check and tell me what instance you are connected to.
```

A healthy response names your instance and its n8n version. That means Claude can now see your workflows.

Troubleshooting the connection

Symptom	Cause	Fix
<code>claude mcp list</code> shows failed	Server cannot start	Run <code>npx -y n8n-mcp</code> directly and read the error
Only <code>search_nodes</code> -type tools exist	No API key reached the server	Check <code>N8N_API_URL</code> / <code>N8N_API_KEY</code> spelling; restart Claude
<code>n8n_health_check</code> returns 401	Key wrong or expired	Re-test with the <code>curl</code> from Part 3
<code>n8n_health_check</code> cannot connect	n8n not running, or wrong port	<code>curl localhost:5678/healthz</code>
Connection drops immediately	Log output corrupting stdio	Set <code>DISABLE_CONSOLE_OUTPUT=true</code>
Tools missing after adding server	Session started before the server	Restart Claude Code
Old version of the server runs	Stale npx cache	<code>npx -y n8n-mcp@latest</code>
Crashes on start	Node too old	<code>node -v</code> , upgrade to 20+

A useful habit: whenever something looks wrong, ask Claude to run `n8n_health_check` first. It distinguishes "not configured" from "configured but unreachable" from "connected fine, your workflow is the problem".

Part 5 — Add the n8n skills plugin (strongly recommended)

The MCP server gives Claude access to n8n. The skills plugin gives it *judgement* — a set of instructions on how to use that access without producing workflows that look right and fail in production.

```
/plugin marketplace add czlonkowski/n8n-skills
/plugin install n8n-mcp-skills
```

Type those in a Claude Code session, not in your shell.

Watch the repository name. The marketplace repo is `czlonkowski/n8n-skills` , while the plugin inside it is called `n8n-mcp-skills` . Using the wrong one for the `marketplace add` step fails with a repository-not-found error.

Restart Claude Code, then confirm with `/plugin` .

What the plugin actually changes

Without it, Claude configures nodes from training data — which is out of date the moment n8n ships a release. With it, Claude follows a protocol:

- 1. Look up the node with `get_node` and read the **live schema** before setting any parameter.
- 2. Prefer expressions over Code nodes, and skip pointless Set nodes.
- 3. Run `validate_workflow` **before** activating anything.
- 4. Re-fetch the workflow afterwards and check the `connections` object, because validation passing does not prove the wires actually landed.

That last point catches a real class of bug: a workflow whose JSON is perfectly valid but whose nodes are not connected to each other. It looks correct on the canvas and does nothing when it runs.

The plugin also carries specialist skills for expressions, error handling, AI agents, sub-workflows, and binary data, which load automatically when the task calls for them.

Part 6 — Optional: the instance-level MCP endpoint

Skip this part if you just want the tutorial workflow. Come back when you want Claude to run workflows that have no webhook trigger.

n8n 2.34 and later ship a second endpoint alongside the Public API: n8n's own MCP server. n8n-mcp will use it when you give it a token, and that unlocks a few things the Public API cannot do:

Capability	What it gives you
<code>n8n_test_workflow</code> in <code>direct</code> mode	Run a Manual or Schedule-triggered workflow — no webhook needed
<code>n8n_explore_node_resources</code>	Resolve real dropdown values (your actual sheet tabs, Slack channels) instead of guessing IDs
<code>n8n_workflow_versions</code> with <code>source: native</code>	n8n's own version history, including edits made in the UI
Data table column operations	Add, rename, and delete columns

Enable it

- 1. In n8n: **Settings** -> **Instance-level MCP**.
- 2. Set **MCP status** to **Enabled**.
- 3. Copy the MCP API key it generates. This is a **separate secret** from your Public API key from Part 3.
- 4. Add it to your MCP config as `N8N_MCP_ACCESS_TOKEN` :

```
"env": {
  "N8N_API_URL": "http://localhost:5678",
  "N8N_API_KEY": "${N8N_API_KEY}",
  "N8N_MCP_ACCESS_TOKEN": "${N8N_MCP_ACCESS_TOKEN}"
}
```

5. Restart Claude Code and run `n8n_health_check` again. It now reports an `officialMcp` block confirming the endpoint is reachable.

The per-workflow switch that confuses everyone

Enabling the endpoint is not enough on its own. **Each workflow** also has to opt in: open the workflow, go to its **Settings**, and turn on **Available in MCP**.

If you forget, calls fail with `WORKFLOW_NOT_EXPOSED`, which is easy to misread as a permissions bug. It is not — it is that one checkbox.

Part 7 — Connect Google Sheets and Gmail to n8n

This is the part most tutorials wave at, and it is where most people get stuck. Take it slowly; you only do it once, and both credentials come from the same Google project.

Both services are free. Gmail's free tier allows roughly 500 sent messages per day, which is far more than a tutorial needs.

Step 7.1 — Create the spreadsheet

In Google Sheets, create a new spreadsheet named **n8n Contacts**. Rename the first tab to **Contacts** — the tab name matters, we reference it later.

Put these four headers in row 1, one per column:

name	email	status	sent_at
------	-------	--------	---------

Then add a single data row, and use **your own email address** for it. Everything in this tutorial sends real mail, so your first test should land in your own inbox:

Ali	you@example.com		
-----	-----------------	--	--

Leave `status` and `sent_at` empty. The workflow fills them in.

Two rules that will save you debugging later:

- **Header names become field names in n8n.** Keep them lowercase, no spaces. `sent_at` is fine; `Sent At` becomes an awkward field to reference.
- **Row 1 must be the headers**, with no blank or title rows above it. n8n reads row 1 to learn the column structure.

Keep the spreadsheet open. You will need its URL shortly.

Step 7.2 — Create a Google Cloud project

Go to **console.cloud.google.com** and sign in with the same Google account that owns the sheet.

1. Click the project dropdown in the top bar, then **New Project**.
2. Name it `n8n-local` and create it.
3. Make sure the project selector now shows `n8n-local`. Everything below happens inside this project, and doing it in the wrong project is the most common way to waste twenty minutes here.

This is free. Google Cloud asks for billing details on some services; nothing in this guide touches those.

Step 7.3 — Enable the two APIs

An OAuth credential only works for APIs you have explicitly switched on.

1. Navigate to **APIs & Services** -> **Library**.
2. Search for **Google Sheets API**, open it, click **Enable**.
3. Go back to the Library, search for **Gmail API**, open it, click **Enable**.

Do this *before* creating credentials. Enabling an API afterwards works, but the change can take a few minutes to propagate, and in the meantime you get permission errors that look like credential problems.

Step 7.4 — Configure the OAuth consent screen

Google will not issue OAuth credentials until it knows what is asking for them.

1. Go to **APIs & Services** -> **OAuth consent screen**.
2. Choose **External** as the user type. (**Internal** only exists if you have Google Workspace, and restricts access to your organisation.)
3. Fill in the required fields: an app name (`n8n local`), your email as the user support email, and your email again as the developer contact. The rest can stay empty.
4. Save and continue through the **Scopes** step without adding anything. n8n requests the scopes it needs at sign-in time.
5. On the **Test users** step, click **Add users** and add **your own Google account**.

That last step is not optional. While the app is in **Testing** status — which it will be, and which is fine — only accounts on the test user list can authorise it. Skip it and sign-in fails with `access_denied` and no useful explanation.

Step 7.5 — Create the OAuth client

1. Go to **APIs & Services** -> **Credentials**.
2. Click **Create Credentials** -> **OAuth client ID**.
3. Application type: **Web application**. Not "Desktop app" — n8n handles the callback over HTTP, so it needs a web client.
4. Name it `n8n`.

5. Under **Authorized redirect URIs**, click **Add URI** and enter exactly:

```
http://localhost:5678/rest/oauth2-credential/callback
```

6. Click **Create**. Google shows your **Client ID** and **Client Secret**. Keep this dialog open, or download the JSON.

The redirect URI is where this goes wrong more than anywhere else. Some notes:

- It must match what n8n sends, character for character. A trailing slash, `https` instead of `http`, or `127.0.0.1` instead of `localhost` all fail.
- `http://localhost` is allowed by Google specifically as an exception for local development. You do not need HTTPS or a public domain.
- If you set `N8N_HOST` or `WEBHOOK_URL` to something other than `localhost:5678`, this URI changes accordingly.
- **The reliable method:** open the credential dialog in n8n (next step). It displays the exact OAuth Redirect URL for your instance. Copy it from there and paste it into Google. Never type it from memory.

Step 7.6 — Create the Google Sheets credential in n8n

1. In n8n, open **Credentials** from the left sidebar, then **Add credential**.
2. Search for and select **Google Sheets OAuth2 API**.
3. At the top of the dialog, n8n shows an **OAuth Redirect URL**. Confirm it matches what you entered in Google. If it does not, fix Google to match n8n.
4. Paste your **Client ID** and **Client Secret**.
5. Click **Sign in with Google**. A popup opens.
6. Choose your Google account.
7. You will see **"Google hasn't verified this app"**. This is expected — it is your own unpublished app. Click **Advanced**, then **Go to n8n local (unsafe)**.
8. Grant the requested permissions.
9. The popup closes and n8n shows **Account connected**.
10. Rename the credential to something you will recognise, and save.

Step 7.7 — Create the Gmail credential

Reuse the same OAuth client. There is no need for a second Google project.

1. **Add credential** -> **Gmail OAuth2 API**.
2. Paste the **same** Client ID and Client Secret.
3. **Sign in with Google**, approve the same way.
4. Save.

You now have two credentials backed by one OAuth client, which is exactly how it should be.

Google credential problems, and what they actually mean

What you see	Real cause	Fix
<code>redirect_uri_mismatch</code>	The URI in Google differs from what n8n sent	Copy the URL from n8n's credential dialog into Google, exactly
<code>access_denied</code> right after choosing your account	Your account is not on the test user list	OAuth consent screen -> Test users -> add yourself
"Google hasn't verified this app"	App is unpublished. Not an error	Advanced -> Go to (unsafe)
<code>Insufficient Permission</code> when the node runs	The API was never enabled, or was enabled after sign-in	Enable the API, then reconnect the credential
Worked yesterday, <code>invalid_grant</code> today	Refresh tokens expire after 7 days while the app is in Testing	Reconnect, or publish the app to Production
Popup blocked or blank	Browser blocking, or wrong <code>WEBHOOK_URL</code>	Allow popups; check n8n's advertised URL
Credentials unreadable after a restart	<code>N8N_ENCRYPTION_KEY</code> changed	Restore the old key, or re-enter credentials

The 7-day token expiry catches almost everyone eventually. While your OAuth app is in Testing status, Google expires refresh tokens after a week and n8n starts failing with `invalid_grant`. Two options: reconnect the credential when it happens, or go to the OAuth consent screen and click **Publish app** to move it to Production. Publishing an app that only requests your own data does not require Google's verification review.

A note on Service Accounts

There is a second way to authenticate Google Sheets that skips the consent screen: a **Service Account**. You create one under **Credentials**, download its JSON key, and share your spreadsheet with the service account's email address — the same way you would share it with a colleague. n8n has a **Service Account** option in the Google Sheets credential for exactly this.

It is genuinely simpler for Sheets, and it never expires. But it does not help here, because **a service account cannot send Gmail** without Workspace domain-wide delegation, which needs a paid Workspace domain. Since our workflow sends email, we use OAuth for both.

Worth remembering for sheet-only automations, though.

Part 8 — Build the workflow through Claude Code

Everything is connected. Now we ask for the workflow in English and watch Claude build it on the instance.

Step 8.1 — Give Claude the details it cannot guess

Two things are unique to you: the spreadsheet and the credentials. Have the spreadsheet URL ready.

Open Claude Code in your project directory and type something like this:

```
Build me an n8n workflow on my local instance.

It should:
1. Start from a Manual Trigger.
2. Read all rows from the "Contacts" tab of this Google Sheet:
  <paste your spreadsheet URL>
3. Only continue for rows where the "status" column is empty.
4. Send each of those contacts a short, friendly email via Gmail,
  addressed to them by name.
5. Write "sent" into the status column and the current time into
  sent_at for that row.

Use my existing Google Sheets and Gmail credentials.
Create it inactive. Do not run it yet.
```

Three details in that prompt do real work, and they are worth copying into your own prompts later:

- **"Use my existing credentials"** stops Claude from inventing credential IDs.
- **"Create it inactive"** guarantees nothing fires while you are still reading it.
- **"Do not run it yet"** matters because Gmail sends real email to real people.

Step 8.2 — What Claude does, and why each step exists

You will see Claude work through a sequence of tool calls. It is worth understanding, because it is the same sequence you should expect for every workflow:

Step	Tool	Purpose
1	search_nodes	Find the right node types for sheets, email, branching
2	get_node	Read the live parameter schema for each one
3	n8n_create_workflow	Create the workflow from complete JSON
4	validate_workflow	Check node configs, required fields, expressions
5	n8n_get_workflow	Re-read it and confirm the connections really landed

Step 2 is the one that separates this from asking a chatbot to write n8n JSON. Node parameters change between versions; a remembered parameter name validates as a plain string and then silently does nothing at runtime. Reading the schema from your instance eliminates that whole failure class.

Step 5 looks redundant and is not. `validate_workflow` confirms the JSON is well-formed. It does not confirm the nodes are wired to each other. A workflow with valid nodes and an empty `connections` object passes validation and does nothing.

Step 8.3 — The workflow you get

Five nodes:

```
Manual Trigger
-> Get Contacts      (Google Sheets: Get Row(s), Contacts tab)
-> Not Yet Sent      (If: status is empty)
-> Send Email        (Gmail: Send a message)
-> Mark As Sent      (Google Sheets: Update Row)
```

The expressions that make it personal:

Field	Expression
Gmail "To"	<code>{{ \$json.email }}</code>
Gmail subject/body greeting	<code>{{ \$json.name }}</code>
Sheets update, <code>status</code>	<code>sent</code>
Sheets update, <code>sent_at</code>	<code>{{ \$now.toISO() }}</code>

`$json` is the current item. `$now` is a Luxon DateTime, so `.toISO()`, `.toFormat('yyyy-MM-dd')`, and date arithmetic all work inline. There is no need for a separate Date & Time node.

Step 8.4 — Three nodes that are deliberately absent

New n8n users reach for these constantly. Understanding why they are not here will make every workflow you build simpler.

No Loop Over Items node. n8n runs nodes once per item automatically. Ten rows come out of the Sheets node, and the Gmail node executes ten times on its own. Adding a loop node to "make it loop" produces nested iteration and a workflow that is harder to reason about.

No Set node. A Set node that feeds exactly one consumer is almost always noise — put the expression directly in the field that needs it. Set nodes earn their place when several downstream nodes share the same computed values.

No Code node. Nothing here needs JavaScript. The order of preference is: plain expression, then an inline function inside Edit Fields, then a Code node only when neither can do the job. Code nodes are the hardest part of a workflow to debug and the easiest to break on upgrade.

Step 8.5 — Run it

Open <http://localhost:5678> and find the new workflow. Read it before running it — click each node and check that the credential dropdown shows your account and the sheet selector shows your spreadsheet.

Then click **Execute Workflow**.

Watch the canvas. Each node turns green with an item count as data flows through. Click any node to see exactly what went in and what came out — this data inspector is the best debugging tool n8n has.

Then check three places:

1. Your inbox: the email arrived.
2. Your spreadsheet: `status` says `sent` and `sent_at` has a timestamp.
3. The **Executions** tab in n8n: a successful run is logged with full data.

You can also ask Claude to inspect the run for you:

```
Show me the last execution of that workflow and tell me
what each node output.
```

Step 8.6 — Iterate: edits are diffs, not rewrites

Ask for a change:

```
Replace the Manual Trigger with a Schedule Trigger that runs
every weekday at 9am Karachi time.
```

Claude uses `n8n_update_partial_workflow`, which applies a small list of operations — remove this node, add that one, rewire this connection — rather than replacing the whole workflow. Your other nodes, their positions, and their credentials are untouched.

This matters when a workflow grows past a handful of nodes. Full rewrites lose things.

Other requests that work well on this workflow:

- "Add an error output on the Gmail node that logs failures to a second sheet tab."
- "Skip rows where the email column is empty."
- "Send at most 20 emails per run."

Part 9 — Verifying, troubleshooting, and staying safe

The verification habit

After any AI-assisted change, check these four things. It takes a minute and catches almost everything:

1. **Validate.** `validate_workflow` returns no errors.
2. **Inspect connections.** Open the canvas — every node is actually wired to the next. No orphans.
3. **Check credentials.** Each node that needs one shows a real account, not a blank dropdown.
4. **Run once with test data.** One row, your own address, before anything real.

Common runtime errors

Error	Cause	Fix
The resource you are requesting could not be found	Wrong sheet ID or tab name	Re-select the document and tab in the node
Insufficient Permission	API not enabled, or credential predates enabling it	Enable the API, reconnect the credential
invalid_grant	Refresh token expired (7-day Testing limit)	Reconnect, or publish the OAuth app
Node outputs 0 items	The If condition matched nothing	Click the If node and inspect its input
Email sent to the wrong person	Expression referencing the wrong node's data	Use <code>\$('Get Contacts').item.json.email</code>
Update writes to the wrong row	Matching column not set	Set the Sheets update node to match on a unique column
Workflow runs but nothing happens	Nodes not connected	Check <code>connections</code> , not just validation

Safety rules

These are short and they matter.

Secrets go in credentials, never in node fields. If you find yourself pasting an API key into a text field, or storing a token in a Set node and referencing it as `{{ $json.token }}` , stop. That is a leak with extra steps. Use n8n's credential system, or the HTTP Request node's built-in credential types.

Never point AI tooling at a production workflow without a backup. Duplicate it first. AI edits are usually right and occasionally surprising, and n8n's undo does not span API-driven changes.

Test sends go to your own address. Every single time. Contact lists are unforgiving.

Treat the n8n API key like a root password. It can read, write, and execute everything on the instance.

Appendix A — Environment variable reference

Variable	Used by	Purpose
N8N_API_URL	n8n-mcp	Base URL of your instance, no trailing slash
N8N_API_KEY	n8n-mcp	Public API key from Settings -> n8n API
N8N_MCP_ACCESS_TOKEN	n8n-mcp	Optional; instance-level MCP endpoint (Part 6)
MCP_MODE	n8n-mcp	stdio for local clients
LOG_LEVEL	n8n-mcp	error keeps stdout clean
DISABLE_CONSOLE_OUTPUT	n8n-mcp	true prevents stdio protocol corruption
N8N_ENCRYPTION_KEY	n8n	Encrypts stored credentials. Never change it
GENERIC_TIMEZONE	n8n	Timezone for Schedule Trigger nodes
WEBHOOK_URL	n8n	Base URL for webhooks and OAuth callbacks
N8N_HOST / N8N_PORT	n8n	Where n8n binds and how it addresses itself

Appendix B — Command cheat sheet

```
# Run n8n
npx n8n

# Is it alive?
curl -s -o /dev/null -w '%{http_code}\n' \
  http://localhost:5678/healthz

# Is the Public API enabled? (401 = yes, needs a key)
curl -s -o /dev/null -w '%{http_code}\n' \
  http://localhost:5678/api/v1/workflows

# Does my key work? (200 = yes)
curl -s -o /dev/null -w '%{http_code}\n' \
  -H "X-N8N-API-KEY: $N8N_API_KEY" \
  http://localhost:5678/api/v1/workflows

# Connect the MCP server
claude mcp add n8n-mcp --scope project \
  -e N8N_API_URL=http://localhost:5678 \
  -e N8N_API_KEY="$N8N_API_KEY" \
  -- npx -y n8n-mcp

# Check it
claude mcp list

# Remove it
claude mcp remove n8n-mcp
```

In-session Claude Code commands:

```
/mcp # servers and their tools
/plugin marketplace add czlonkowski/n8n-skills
/plugin install n8n-mcp-skills
```

Appendix C — Useful prompts

Run `n8n_health_check` and tell me which instance you are on.

List my workflows and show which are active.

Look up the Google Sheets node and tell me what operations it supports.

Validate workflow `<id>` and explain any warnings.

Show me the last failed execution and what caused it.

Find a template for `<task>` and adapt it to my instance.